Datalogger SPQ — Actualizar el firmware con una microSD

Manual de usuario — firmware FWDLGARM 0.0.24 o posterior, placa SPQ_ARM_R1

Spymovil — Desarrollo de firmware

4 de noviembre de 2026 — rev. 2

Índice

1. Para qué sirve **2**

2. Lo que hace falta **2**
2.1. Cómo se genera el archivo . . . 2

3. Convención de nombres: `FIRMWARE.BIN` y `FIRMWARE.OK` **2**

4. Procedimiento A — automático (el normal en campo) **3**
4.1. Qué ocurre durante esos 20 segundos . . . 3
4.2. Cómo saber que funcionó, sin consola . . . 3
4.3. Cómo saber que funcionó, con consola . . . 3

5. Procedimiento B — a mano, por consola **4**
5.1. Paso 1: `fw` — examinar sin instalar . . . 4
5.2. Paso 2: `fw grabar` — escribir el banco de respaldo . . . 4
5.3. Paso 3: `fw commit aplicar` — instalarlo . . . 5

6. La marca de “firmware a prueba” **5**
6.1. Qué problema resuelve . . . 5
6.2. Dónde se guarda . . . 5
6.3. Dónde se ve . . . 5
6.4. Cómo opera . . . 6
6.5. Dos límites de la marca que hay que conocer . . . 6

7. Los bancos de memoria **7**
7.1. Qué son . . . 7
7.2. Cómo saber qué hay en cada uno . . . 7

8. Casos de uso **8**
8.1. 1. Actualizar un equipo en campo . . . 8
8.2. 2. Actualizar varios equipos con la misma tarjeta . . . 8
8.3. 3. Volver a una versión anterior . . . 8
8.4. 4. Un equipo nuevo, recién fabricado . . . 8
8.5. 5. Un equipo que no se puede visitar seguido . . . 8

9. Mensajes de error: qué dice el equipo y qué hacer **8**

10. Fallas y recuperación **9**
10.1 Si se corta la alimentación en medio de la actualización . . . 9
10.2 Si el equipo no arranca después de actualizar . . . 9
10.3 Si la consola no contesta justo después de usar el programador . . . 10

11 Lo que este mecanismo garantiza, y lo que no **10**
11.1 Garantiza 10
11.2 No garantiza 10

12 Resumen rápido **10**

1. Para qué sirve

Permite **actualizar el firmware de un datalogger en campo con una tarjeta microSD**, sin programador, sin computadora y sin abrir el gabinete. El equipo lee la tarjeta al encenderse, y si encuentra un firmware válido y distinto del que tiene, lo instala y se reinicia solo.

El mecanismo está pensado para que lo use un técnico de campo. Toda la verificación la hace el equipo: si el archivo está dañado, es de otro producto, o está incompleto, **no lo instala y lo dice**.

> **Lo que este mecanismo no es.** No es una actualización por aire (OTA): el archivo lo lleva alguien en una tarjeta. La infraestructura está preparada para que algún día el modem deje el archivo en la tarjeta y el resto funcione igual, pero eso hoy no existe.

2. Lo que hace falta

Una **tarjeta microSD**	la misma que el equipo usa para datos. No hace falta que esté vacía, ni formatearla en una PC: el equipo la formatea con `fs sd format borrar` si hiciera falta
El archivo **`FIRMWARE.BIN`**	en la **raíz** de la tarjeta (no en una carpeta)
Nada más	no hace falta programador, ni cable SWD, ni consola, si se usa el procedimiento automático

2.1. Cómo se genera el archivo

Lo genera el equipo de desarrollo con una herramienta del propio proyecto:

```
python3 Tools/mkfw.py Debug/FWDLGARM_R1.elf -o FIRMWARE.BIN
```

La herramienta toma el firmware compilado y le agrega adelante una cabecera de 64 bytes con la versión, el modelo de hardware, el tamaño y una **suma de verificación (CRC-32)** de la imagen completa.

La versión y el modelo no se escriben a mano: la herramienta los saca del propio código fuente. Eso es deliberado, para que no pueda existir un archivo que diga ser una versión que no es.

La cabecera es legible con un visor hexadecimal, así que se puede verificar una tarjeta sin tener el equipo delante:

```
$ xxd -l 64 FIRMWARE.BIN
00000000: 5350 5146 0100 4000 302e 302e 3233 0000  SPQF..@.0.0.23..
00000010: 0000 0000 0000 0000 5350 515f 4152 4d5f  ........SPQ_ARM_
00000020: 5231 0000 0000 0000 0000 0000 30e8 0300  R1..........0...
```

Se leen a simple vista la firma `SPQF`, la versión (`0.0.23`) y el hardware (`SPQ_ARM_R1`).

3. Convención de nombres: `FIRMWARE.BIN` y `FIRMWARE.OK`

Es la única convención que hay que recordar, y son dos nombres:

Nombre	Significado
`FIRMWARE.BIN`	*"instalá esto"*. Es el único nombre que el equipo busca al arrancar
`FIRMWARE.OK`	*"esto ya está instalado en el equipo"*. El equipo lo renombra así después de aplicarlo, y a partir de ese momento lo ignora

El renombrado lo hace el equipo, no el técnico. Es lo que garantiza que una actualización se aplique **una sola vez**: si el archivo siguiera llamándose `FIRMWARE.BIN`, el equipo podría volver a instalarlo en cada arranque.

> **IMPORTANTE.** Para volver a instalar el mismo archivo —por ejemplo, para pasar la misma actualización a un segundo equipo con la misma tarjeta— hay que **renombrar `FIRMWARE.OK` a `FIRMWARE.BIN`** en una computadora. El equipo no tiene un comando para hacerlo, a propósito: un comando así invita a reinstalar sin querer.

Los nombres son de 8+3 caracteres (formato `8.3`) porque el equipo no usa nombres largos. Cualquier otro nombre es ignorado por el procedimiento automático; se puede examinar a mano con `fw <nombre>`, pero el automático sólo mira `FIRMWARE.BIN`.

4. Procedimiento A — automático (el normal en campo)

Es el procedimiento recomendado y no necesita consola.

1. **Copiar `FIRMWARE.BIN` a la raíz de la tarjeta**, en una computadora.
2. **Insertar la tarjeta en el datalogger.**
3. **Reiniciar el equipo** (cortar y reponer la alimentación, o el comando `reset` si hay consola).
4. **Esperar unos 20 segundos sin desconectar la alimentación.**
5. Listo. El equipo arranca ya con el firmware nuevo.

4.1. Qué ocurre durante esos 20 segundos

Tiempo	Qué hace el equipo
0 – 2 s	lee el archivo completo y verifica su CRC
2 – 15 s	escribe el firmware en el **banco de respaldo** y lo vuelve a leer para verificarlo
15 s	renombra el archivo a `FIRMWARE.OK`
15 – 18 s	copia el firmware al banco de arranque
18 s	se reinicia solo

> **ATENCIÓN: no cortar la alimentación durante el paso de los 15 a los 18 segundos.** Es el único tramo en que el equipo está reescribiendo el firmware que ejecuta. En el capítulo de fallas se explica qué pasa si se corta igual, y cómo se recupera; está previsto, pero conviene evitarlo.

4.2. Cómo saber que funcionó, sin consola

Hay dos señales, y cualquiera de las dos alcanza:

1. **La tarjeta**: el archivo pasó a llamarse `FIRMWARE.OK`.
2. **El servidor**: el equipo informa su versión en cada sesión (campo `VER=`).

4.3. Cómo saber que funcionó, con consola

El banner de arranque dice la versión y la fecha de compilación:

```
FWDLGARM_R1 0.0.23 - consola TERM
compilado Oct 31 2026 10:40:55
```

Y aparece un aviso que se explica en el capítulo siguiente:

```
[!] FIRMWARE A PRUEBA: arranque 1 de 5 sin confirmar
    se confirma solo cuando una sesion llegue a TRANSMITIR
    (PING + CONF_ALL: eso ejercita modem, I2C, EEPROM, RTC y ADC)
    la version anterior era 0.0.22
```

5. Procedimiento B — a mano, por consola

Sirve para hacer lo mismo paso a paso, viendo cada resultado. Es lo que conviene usar en el banco, o en campo cuando algo no salió y hay que entender qué pasó.

```
cmd>fw                      <- ¿hay un firmware nuevo? Lo examina. NO lo instala
cmd>fw grabar               <- lo escribe en el banco de respaldo y lo verifica
cmd>fw commit aplicar       <- lo instala y reinicia el equipo
```

5.1. Paso 1: `fw` — examinar sin instalar

Este comando **no escribe un solo byte** de memoria de programa. Lee el archivo, verifica la cabecera, recalcula el CRC de la imagen completa y compara la versión.

```
cmd>fw
FW:: el que esta CORRIENDO
  version    : 0.0.22  (Oct 30 2026 23:02:47)
  hardware   : SPQ_ARM_R1
  banco      : 512 KB (la flash son 2 bancos de ese tamaño)

FW:: LOS BANCOS
  corriendo de  : BANCO 1
  arrancara de  : BANCO 1   (option byte BFB2)

FW:: el BANCO 2 (el respaldo, en 0x08080000)
  stack pointer : 0x20040000
  vector reset  : 0x08018A49
  ==> PARECE un firmware: SP en SRAM1, reset en flash e impar

FW:: mirando la microSD (FIRMWARE.BIN)...
  resultado  : hay un firmware nuevo y valido
  el archivo dice:
    version  : '0.0.20'
    hardware : 'SPQ_ARM_R1'
    imagen   : 256080 bytes
    CRC      : declarado 0x24C473F4 / leido 0x24C473F4  OK
  tardo      : 1996 ms

  ==> HAY UN FIRMWARE NUEVO Y VALIDO ('0.0.20' contra '0.0.22')
```

`fw <nombre>` examina otro archivo en vez de `FIRMWARE.BIN`. Sirve para revisar un `FIRMWARE.OK`, o un archivo que alguien dejó con otro nombre, sin renombrar nada.

5.2. Paso 2: `fw grabar` — escribir el banco de respaldo

Escribe la imagen en el banco 2 y **la vuelve a leer de la memoria** para verificarla. Tarda unos 13 segundos.

```
cmd>fw grabar
FW:: borrando el banco 2 (126 paginas de 2048 B)...
FW:: escribiendo 256080 bytes...
```

```
  ==> EL BANCO 2 QUEDO ESCRITO Y VERIFICADO
      CRC releido de la flash: 0x24C473F4  (igual al del archivo)
```

Al terminar este paso el equipo sigue arrancando del firmware viejo. Si algo falla acá, no pasa nada: el firmware en uso no se tocó.

5.3. Paso 3: `fw commit aplicar` — instalarlo

Copia el banco 2 al banco de arranque y reinicia. **No vuelve**: la consola se corta y reaparece con el banner del firmware nuevo.

```
cmd>fw commit aplicar
FW:: COMMIT — el banco 2 pasa a ser el firmware del equipo
  aplicando... (la consola se corta aca)

FWDLGARM_R1 0.0.20 - consola TERM
```

La palabra **`aplicar` es obligatoria**: `fw commit` solo no hace nada y muestra la ayuda. Es una red deliberada, porque este paso reescribe el firmware en uso.

6. La marca de “firmware a prueba”

6.1. Qué problema resuelve

Una actualización puede salir mal de dos maneras muy distintas:

El firmware nuevo **no arranca**	se nota enseguida: el equipo no da señales
El firmware nuevo **arranca pero no funciona**	no transmite, no lee un sensor, se reinicia cada tanto. **Esto es lo peligroso**: desde afuera el equipo parece bien, el técnico se va, y pueden pasar meses sin que nadie se entere

La marca existe para el segundo caso. Hace que el equipo lo diga en voz alta, en cada arranque, hasta que se compruebe que el firmware nuevo realmente sirve.

6.2. Dónde se guarda

En la **memoria EEPROM externa del equipo**, en la dirección `0x00400`. Son 22 bytes: una firma, la versión que se estaba reemplazando, un contador de arranques y una suma de verificación.

Es la misma memoria donde vive la configuración del equipo, y eso es a propósito: si esa memoria fallara, el equipo lo grita por todos lados (la configuración se iría a valores de fábrica y el servidor pediría reconfigurar todo), así que **no agrega un punto de falla silencioso**.

La marca **sobrevive a los reinicios y a los cortes de alimentación**, porque la EEPROM no se borra.

6.3. Dónde se ve

Hay tres lugares, y el primero no hay que pedirlo:

1. El banner de arranque, en cada arranque:

```
[!] FIRMWARE A PRUEBA: arranque 1 de 5 sin confirmar
    se confirma solo cuando una sesion llegue a TRANSMITIR
    (PING + CONF_ALL: eso ejercita modem, I2C, EEPROM, RTC y ADC)
    la version anterior era 0.0.22
```

2. El comando `fw prueba`:

```
cmd>fw prueba
FW:: sin marca: este firmware esta CONFIRMADO
     (o nunca se aplico uno con 'fw commit')
```

3. El registro de eventos de la microSD (`EVENTOS.TXT`, si está habilitado), con una línea por arranque:

```
FW: a prueba, arranque 1 de 5 sin confirmar
```

6.4. Cómo opera

1. **Se pone sola** cuando se instala un firmware, tanto en el procedimiento automático como en el manual. Guarda **la versión que se está reemplazando**, para poder nombrarla en los avisos.
2. **Cada arranque suma uno** al contador y lo anuncia.
3. **Se borra sola** cuando el equipo completa una sesión con el servidor que **llega a transmitir**, o sea cuando el `PING` volvió y el frame de configuración se procesó.
4. **Al llegar a 5 arranques sin confirmar**, el aviso se vuelve explícito:

   ```
   SE PASO DEL LIMITE: este firmware no logro una sesion.
      revisar el enlace, y si no es eso, volver al anterior
      con 'fw grabar' + 'fw commit aplicar' desde la tarjeta.
   ```

6.4.1. Por qué el criterio es “una sesión con el servidor”

Porque es la prueba más completa que el equipo puede hacer, y además es trabajo que hace igual. Una sesión que llega a transmitir ejercita **el modem, el bus I2C, la memoria EEPROM, el reloj de tiempo real y el conversor analógico** de una sola vez. Cualquier autodiagnóstico que escribiéramos probaría menos.

> **ATENCIÓN: la condición es “llegó a transmitir”, no “el servidor dijo que todo coincide”.** La marca se borra cuando el equipo completó el intercambio de configuración y pasa a enviar datos — **también si el servidor pidió reconfigurar algún bloque**. Es deliberado: lo que se quiere comprobar es que el equipo *funciona*, y para eso alcanza con que la sesión haya llegado hasta ahí. Hasta el 2026-11-04 el equipo anunciaba en su banner la condición más estricta (`CONFIG=OK`), que prometía más de lo que el programa exige; ya está corregido.

6.4.2. Borrarla a mano

```
cmd>fw prueba confirmar
FW:: marca BORRADA: el firmware queda confirmado
```

Sirve en el banco, o en campo cuando el equipo todavía no tiene cobertura y uno ya comprobó por otros medios que el firmware anda. Es inofensivo: lo único que hace es apagar el aviso.

6.5. Dos límites de la marca que hay que conocer

> **La marca avisa, pero no vuelve atrás sola.** Al pasarse de los 5 arranques el equipo lo informa, y nada más. El motivo es concreto: después de instalar, **los dos bancos tienen el firmware nuevo**, así que la imagen anterior ya no existe dentro del equipo. Volver atrás requiere la tarjeta con el firmware anterior. Prometer un retorno automático sin tener la imagen sería prometer algo que no se puede cumplir.
>
> **Bajar un firmware con el programador no borra la marca.** Si un equipo se reprograma por cable SWD mientras tenía una marca puesta, el firmware nuevo se va a anunciar “a prueba” de una actualización que no hizo. Es inofensivo y se arregla con `fw prueba confirmar`. No se puede evitar desde el firmware: un programa no tiene forma de saber cómo llegó a la memoria.

7. Los bancos de memoria

7.1. Qué son

La memoria de programa del equipo son **1024 KB divididos en dos bancos de 512 KB**:

```
BANCO 1   0x08000000 .. 0x0807FFFF   el firmware que se ejecuta
BANCO 2   0x08080000 .. 0x080FFFFF   el respaldo: acá se escribe el nuevo
```

El firmware completo ocupa unos 258 KB, o sea la mitad de un banco.

Para qué sirve tener dos. El firmware nuevo se escribe en el banco 2 **mientras el equipo sigue funcionando normalmente con el banco 1**. Si esa escritura falla, no pasó nada: el equipo sigue con su firmware de siempre. Sólo cuando el banco 2 está escrito *y verificado* se lo copia al banco 1.

7.2. Cómo saber qué hay en cada uno

El comando `fw` lo informa siempre, sin que haya que pedirlo:

```
FW:: LOS BANCOS
  corriendo de  : BANCO 1
  arrancara de  : BANCO 1   (option byte BFB2)

FW:: el BANCO 2 (el respaldo, en 0x08080000)
  stack pointer : 0x20040000
  vector reset  : 0x08018A49
  ==> PARECE un firmware: SP en SRAM1, reset en flash e impar
```

Cómo leer eso:

Línea	Qué significa
`corriendo de`	de qué banco se está ejecutando el firmware **ahora mismo**
`arrancara de`	de qué banco va a arrancar la próxima vez
`stack pointer` y `vector reset`	las dos primeras palabras del banco 2. Son lo único que un microcontrolador necesita para arrancar
`PARECE un firmware`	esas dos palabras tienen valores creíbles para un firmware
`esta EN BLANCO`	el banco 2 nunca se escribió: no hay nada para instalar
`NO parece un firmware`	hay algo escrito, pero no parece un programa. **Avisa, no impide**: el comando se puede ejecutar igual

El contenido exacto de cada banco no se puede ver desde la consola, y no hace falta: lo que importa es la versión que el equipo informa, y si el banco 2 verifica su CRC. Para una inspección byte a byte hace falta un programador.

> **ADVERTENCIA: no usar `fw boot 2`.** Ese comando existe para cambiar el banco de arranque, y **en esta placa no funciona**: el microcontrolador queda sin ejecutar nada y hace falta un programador para rescatarlo. La ayuda del comando (`help fw`) lo advierte. El procedimiento correcto es `fw commit aplicar`, que copia en vez de cambiar de banco.

8. Casos de uso

8.1. 1. Actualizar un equipo en campo

Procedimiento A. Es el caso normal.

8.2. 2. Actualizar varios equipos con la misma tarjeta

Después del primer equipo, el archivo quedó como `FIRMWARE.OK`. Para el segundo equipo hay que **renombrarlo a `FIRMWARE.BIN`** en una computadora.

Alternativa más cómoda: llevar **una tarjeta por equipo**, o copiar el archivo de nuevo entre equipos si se lleva una laptop.

8.3. 3. Volver a una versión anterior

Funciona igual que una actualización, y no hace falta ningún truco: el equipo acepta **cualquier versión distinta de la que tiene**, no solamente una superior.

Esto es deliberado. Si sólo aceptara versiones más nuevas, volver atrás obligaría a fabricar un número de versión falso — y volver atrás es exactamente lo que se necesita cuando una actualización salió mal.

> Está probado en banco: un equipo con `0.0.22` se bajó a `0.0.20` con este mecanismo.

8.4. 4. Un equipo nuevo, recién fabricado

El primer firmware se baja con programador, como siempre. Este mecanismo es para los que vienen después.

8.5. 5. Un equipo que no se puede visitar seguido

Conviene **dejar la tarjeta con el firmware anterior** después de actualizar (renombrada a `FIRMWARE.BIN`, no a `.OK`) sólo si se quiere que el equipo vuelva atrás en el próximo reinicio. En cualquier otro caso, dejarla como `FIRMWARE.OK`: así el equipo no la toca.

9. Mensajes de error: qué dice el equipo y qué hacer

Todos estos mensajes significan que **el equipo no instaló nada y sigue funcionando con su firmware de siempre**. La memoria de programa no se tocó.

El equipo dice	Qué pasó	Qué hacer
`no hay microSD`	la tarjeta no está puesta, o no se detecta	revisar que esté bien insertada
`la tarjeta NO TIENE ese archivo`	no hay `FIRMWARE.BIN` en la raíz	verificar el nombre y que no esté dentro de una carpeta. Si dice `FIRMWARE.OK`, ya estaba instalado
`no se pudo leer el archivo`	la tarjeta falla, o el sistema de archivos está dañado	probar con otra tarjeta. Si se repite, formatear con `fs sd format borrar` y volver a copiar
`no es un firmware de este mecanismo (magic)`	el archivo no es un firmware: se copió otra cosa	volver a generar el archivo con `mkfw.py`
`el header esta CORRUPTO`	la cabecera no verifica: la copia salió mal	volver a copiar el archivo a la tarjeta

El equipo dice	Qué pasó	Qué hacer
`es el firmware de OTRO producto`	el archivo es de otro modelo de equipo	conseguir el archivo correcto. **El equipo nunca va a instalar esto**
`el largo no cierra, o no entra en el banco`	el archivo está truncado, o es más grande que un banco	la copia quedó incompleta: volver a copiarlo
`la imagen esta CORRUPTA`	la cabecera está bien pero la imagen no verifica su CRC	volver a copiar. Si se repite con la misma tarjeta, la tarjeta está fallando
`es la misma version que ya esta corriendo`	no hay nada que hacer: ya tiene esa versión	correcto, no es un error
`formato de header desconocido`	el archivo lo generó una herramienta más nueva que este firmware	actualizar primero a una versión intermedia

En el procedimiento automático, todos estos casos salen por consola y quedan en el registro de eventos, y el equipo arranca normalmente. Las dos únicas situaciones en que el automático se queda **callado** son *no hay tarjeta* y *la tarjeta no tiene el archivo*, porque son el caso normal de todos los arranques de todos los equipos.

10. Fallas y recuperación

10.1. Si se corta la alimentación en medio de la actualización

Depende del momento, y el mecanismo está diseñado para que casi todo el tiempo sea inofensivo:

Si se corta…	Qué queda
mientras lee la tarjeta o escribe el banco de respaldo (0 – 15 s)	**el firmware viejo, intacto.** El equipo arranca normal
mientras copia al banco de arranque (unos 2,7 s)	el equipo **no arranca su firmware**, pero queda en un estado recuperable: arranca el cargador interno del microcontrolador. Ver abajo
en la última fracción de segundo	el firmware nuevo, ya instalado

La ventana realmente delicada es de **82 microsegundos**: el instante en que se escribe la primera palabra del firmware. Está hecho así a propósito —se escribe última, justo por esto—.

10.2. Si el equipo no arranca después de actualizar

Primero hay que distinguir dos situaciones, porque se arreglan distinto:

Caso 1: el equipo no da ninguna señal de vida. Ni banner por consola, ni LED.

Es el caso de un corte de alimentación en el tramo malo, o de un firmware que no arranca. El equipo **no está perdido**: el microcontrolador tiene un cargador interno que toma el control automáticamente cuando la memoria de programa está vacía. La recuperación necesita un técnico con programador:

```
STM32_Programmer_CLI -c port=SWD freq=950 mode=UR -w FWDLGARM_R1.elf
STM32_Programmer_CLI -c port=SWD freq=950 mode=HOTPLUG -hardRst
```

Caso 2: el equipo arranca y funciona, pero no transmite (o falla algo).

Acá sirve la marca: el equipo lo va a estar anunciando en cada arranque. La recuperación es volver al firmware anterior **con el mismo procedimiento de este manual**, poniendo en la tarjeta el archivo de la versión vieja.

Por eso conviene conservar el archivo de la versión anterior. Es el camino de retorno, y no requiere programador.

10.3. Si la consola no contesta justo después de usar el programador

No es una falla del equipo. Al bajar firmware con la herramienta de línea de comandos, el microcontrolador queda retenido en reset y hay que liberarlo:

```
STM32_Programmer_CLI -c port=SWD freq=950 mode=HOTPLUG -hardRst
```

El síntoma es exactamente *"el equipo parece muerto"*: no sale ni el banner. Es un detalle de la herramienta, no del firmware.

11. Lo que este mecanismo garantiza, y lo que no

11.1. Garantiza

- **Que no se instala un archivo dañado.** La imagen entera se verifica por CRC antes de escribir nada, y el banco de respaldo se vuelve a leer de la memoria para comprobar que quedó bien escrito.
- **Que no se instala el firmware de otro producto.** El modelo de hardware viaja en la cabecera.
- **Que una actualización se aplica una sola vez**, por el renombrado a `FIRMWARE.OK`.
- **Que un fallo al escribir no deja el equipo inservible.** Mientras se escribe el banco de respaldo, el firmware en uso no se toca.
- **Que un firmware que arranca y no funciona se hace visible**, por la marca.

11.2. No garantiza

- **No protege contra un firmware que no arranca.** Si el programa nuevo falla antes de inicializar, no hay software que pueda ayudar: hace falta programador. El mecanismo no promete lo contrario.
- **No vuelve atrás solo.** Ver los límites de la marca.
- **No valida el origen del archivo.** El CRC detecta corrupción, no manipulación intencional. Quien tenga acceso físico al equipo puede reprogramarlo por cable de todas formas, así que una firma criptográfica no agregaría seguridad real hoy.

12. Resumen rápido

Para actualizar:

1. Copiar `FIRMWARE.BIN` a la raíz de la microSD.
2. Insertar la tarjeta.
3. Reiniciar el equipo y esperar 20 segundos sin cortar la alimentación.
4. Verificar: el archivo pasó a `FIRMWARE.OK`, y el equipo informa la versión nueva.

Si algo no salió:

- Con consola: `fw` dice exactamente qué pasó.
- Sin consola: el registro de eventos de la tarjeta (`EVENTOS.TXT`) tiene la misma información.
- El equipo nunca queda peor que antes salvo que se corte la alimentación en los últimos 3 segundos.

Comandos de consola, todos juntos:

```
fw                      examinar la tarjeta, sin instalar nada
fw <nombre>             examinar otro archivo
fw grabar               escribir el banco de respaldo y verificarlo
fw commit aplicar       instalarlo y reiniciar
fw prueba               ver la marca de "a prueba"
```

```
fw prueba confirmar     borrar la marca a mano
help fw                 la ayuda del comando
```